University of BRISTOL

LECTURE 5

Backpropagation

Foundations of Artificial Intelligence

SEMTM0051 · 2026/27

Lecturer: Dr Roberto Colomboni



In this lecture…

How can we efficiently calculate the derivatives of the empirical loss with respect to every weight and bias of a neural network?

Backpropagation in a scalar network

The same calculation with vectors

Extensions and computational cost

We have a loss. We need its gradient.

The forward pass evaluates the loss at the current parameters.

$$\underbrace{\mathbf{x}}_{\text{input}} \longrightarrow \underbrace{f_{\boldsymbol{\theta}}(\mathbf{x})}_{\text{prediction/score}} \longrightarrow \underbrace{\mathcal{L}(\boldsymbol{\theta})}_{\text{loss}}$$

When trying to minimise the loss using gradient descent,
we need its gradient with respect to the network parameters.

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}\left(\boldsymbol{\theta}^{(t)}\right)$$

How do we find a derivative for every weight and bias?

Example: a scalar network

Architecture: **1 → 1 → 1 → 1**

Two hidden layers and an affine output. One neuron in each layer.



We write $\phi_0(x) = x$. The index r identifies the layer.

The parameter vector collects all weights and biases.

$$\boldsymbol{\theta} = (b_1, w_1, b_2, w_2, b_3, w_3)^\mathsf{T} \in \mathbb{R}^6$$

Preactivation

$$a_1(x) = \phi_0(x)w_1 + b_1$$

$$a_2(x) = \phi_1(x)w_2 + b_2$$

Activation

$$\phi_1(x) = \mathrm{ReLU}\big(a_1(x)\big)$$

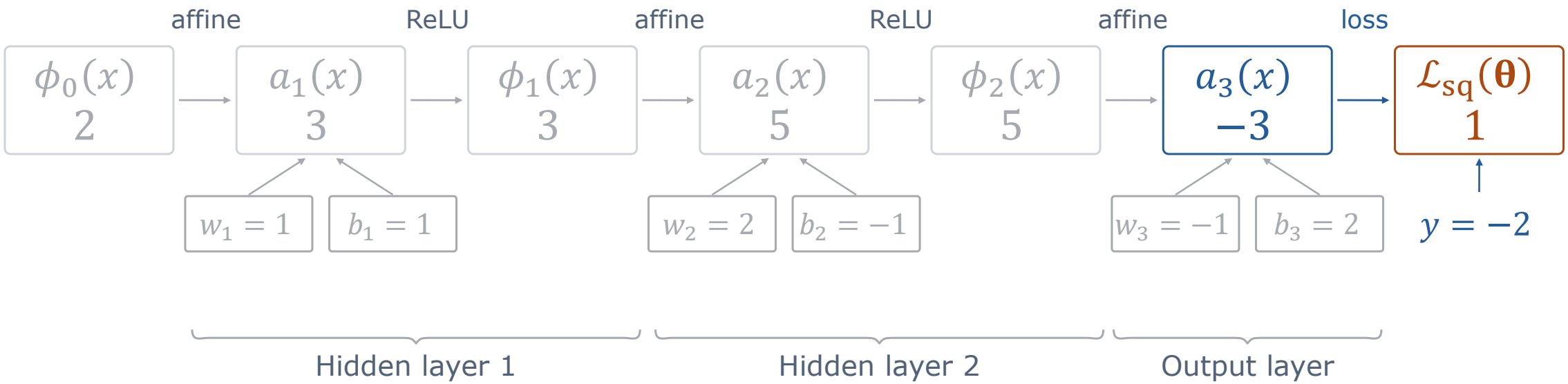
$$\phi_2(x) = \mathrm{ReLU}\big(a_2(x)\big)$$

$$a_3(x) = \phi_2(x)w_3 + b_3 = f_{\boldsymbol{\theta}}(x)$$

First, save the forward values

We consider regression with the squared loss.
One fixed observation: input $x = 2$, target $y = -2$.

For example, consider $\boldsymbol{\theta}^{(t)} = (b_1, w_1, b_2, w_2, b_3, w_3)^\mathsf{T} = (1,1,-1,2,2,-1)^\mathsf{T}$.



$$\mathcal{L}_{\mathrm{sq}}\left(\boldsymbol{\theta}^{(t)}\right) = (a_3(x) - y)^2 = (-3 + 2)^2 = 1$$

Keep these values: the backward calculation will use them to compute the gradient

Six parameters, six partial derivatives

The gradient we want to compute

$$\nabla \mathcal{L}_{\text{sq}}\left(\boldsymbol{\theta}^{(t)}\right) =$$

$$\left(\frac{\partial \mathcal{L}_{\text{sq}}}{\partial b_1}\left(\boldsymbol{\theta}^{(t)}\right), \frac{\partial \mathcal{L}_{\text{sq}}}{\partial w_1}\left(\boldsymbol{\theta}^{(t)}\right), \frac{\partial \mathcal{L}_{\text{sq}}}{\partial b_2}\left(\boldsymbol{\theta}^{(t)}\right), \frac{\partial \mathcal{L}_{\text{sq}}}{\partial w_2}\left(\boldsymbol{\theta}^{(t)}\right), \frac{\partial \mathcal{L}_{\text{sq}}}{\partial b_3}\left(\boldsymbol{\theta}^{(t)}\right), \frac{\partial \mathcal{L}_{\text{sq}}}{\partial w_3}\left(\boldsymbol{\theta}^{(t)}\right)\right)^{\mathsf{T}}$$

The idea of backpropagation

Compute the loss derivatives with respect to the last layer's parameters first.

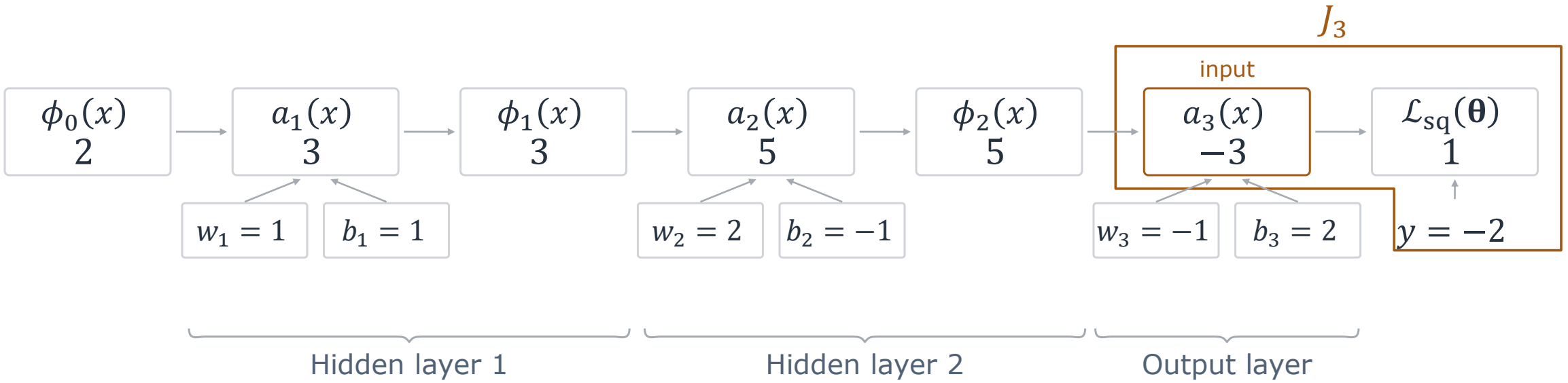
**Then work backwards,
reusing the derivatives already computed.**

How is this achieved in practice?

Start from the last layer

Layer 3: write the loss of the network with parameters $\boldsymbol{\theta}$ on the observation (x, y) as a function of the current layer preactivation $a_3(x)$.

We leave the dependence on the parameters and target implicit and write J_r.

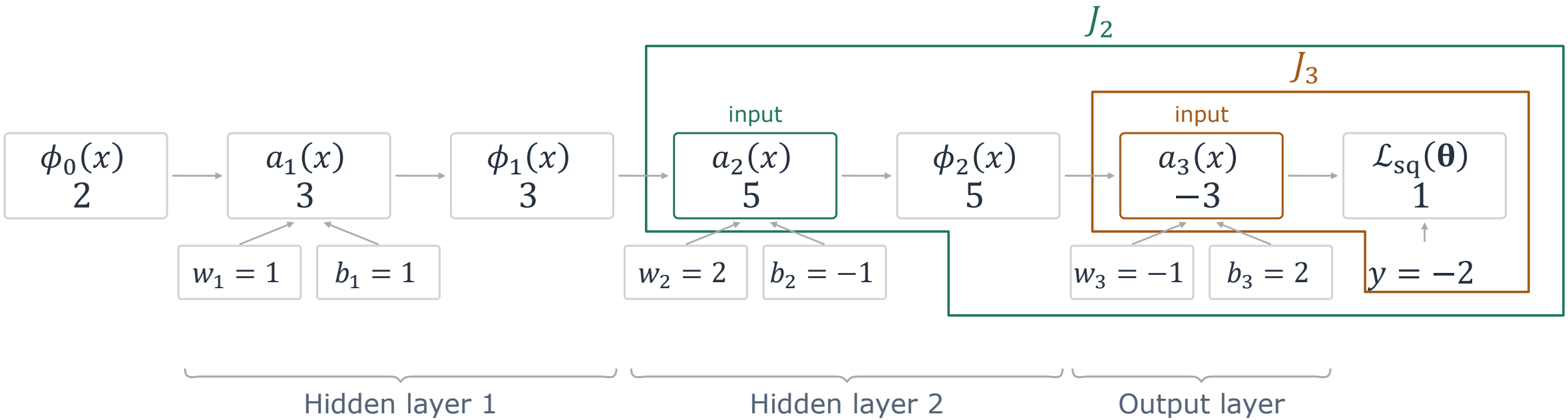


$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = J_3\big(a_3(x)\big), \text{ where } J_3(u) = (u - y)^2$$

J_3 uses y, but not any parameters from the current or previous layers.

Move the starting point one layer back

Layer 2: write the loss of the network with parameters $\boldsymbol{\theta}$ on the observation (x, y) as a function of the current layer preactivation $a_2(x)$.

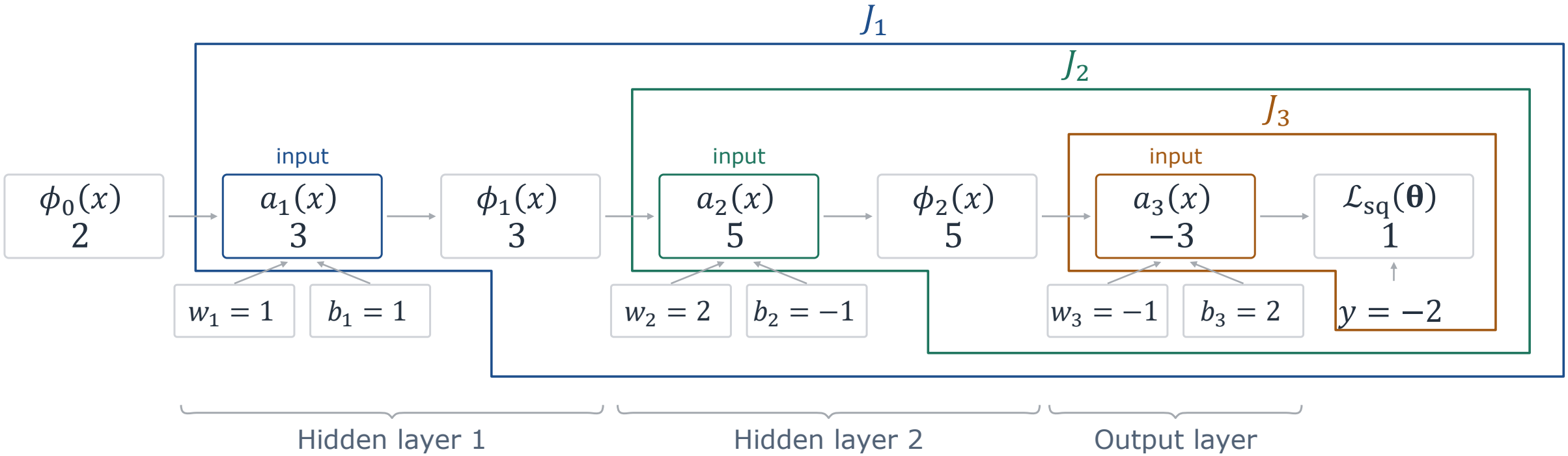


$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = J_2\big(a_2(x)\big), \text{ in fact } J_2\big(a_2(x)\big) = J_3(\underbrace{\text{ReLU}\big(a_2(x)\big)w_3 + b_3}_{a_3(x)})$$

J_2 uses w_3, b_3, y, but not any parameters from the current or previous layers.

Another layer back

Layer 1: write the loss of the network with parameters $\boldsymbol{\theta}$ on the observation (x, y) as a function of the current layer preactivation $a_1(x)$.



$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = J_1\big(a_1(x)\big), \text{ in fact } J_1\big(a_1(x)\big) = J_2(\underbrace{\text{ReLU}\big(a_1(x)\big)w_2 + b_2}_{a_2(x)})$$

J_1 uses w_2, b_2, w_3, b_3, y, but not any parameters from the current or previous layers.

The chain rule at the output layer

Let's start from the 3rd layer

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = J_3\big(a_3(x)\big)$$

where

$$J_3(u) = (u - y)^2 \qquad a_3(x) = \phi_2(x)w_3 + b_3$$

Neither J_3 nor $\phi_2(x)$ depends on w_3 or b_3

$$\frac{\partial \mathcal{L}_{\text{sq}}}{\partial b_3}(\boldsymbol{\theta}) = J_3'\big(a_3(x)\big)\,\frac{\partial a_3(x)}{\partial b_3} = J_3'\big(a_3(x)\big) \cdot 1$$

$$\frac{\partial \mathcal{L}_{\text{sq}}}{\partial w_3}(\boldsymbol{\theta}) = J_3'\big(a_3(x)\big)\,\frac{\partial a_3(x)}{\partial w_3} = J_3'\big(a_3(x)\big) \cdot \phi_2(x)$$

We still need the shared factor $J_3'\big(a_3(x)\big)$ in both expressions.

Completing the output-layer derivatives

We need the factor shared by both parameter derivatives.

$$J_3(u) = (u - y)^2$$

Differentiate with respect to the argument, keeping the target fixed.

$$J_3'(u) = 2(u - y)$$

Evaluate at the preactivation saved during the forward pass.

$$J_3'\big(a_3(x)\big) = 2(a_3(x) - y)$$

Hence, we have computed

$$\frac{\partial \mathcal{L}_{\text{sq}}}{\partial b_3}(\boldsymbol{\theta}) = J_3'\big(a_3(x)\big) \qquad \frac{\partial \mathcal{L}_{\text{sq}}}{\partial w_3}(\boldsymbol{\theta}) = J_3'\big(a_3(x)\big) \cdot \phi_2(x)$$

We will reuse $J_3'\big(a_3(x)\big)$ to calculate $J_2'\big(a_2(x)\big)$.

The chain rule at the second layer

Let's move to the 2nd layer

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = J_2\big(a_2(x)\big)$$

where

$$J_2(u) = J_3(\text{ReLU}(u)w_3 + b_3) \qquad a_2(x) = \phi_1(x)w_2 + b_2$$

Neither J_2 nor $\phi_1(x)$ depends on w_2 or b_2

$$\frac{\partial \mathcal{L}_{\text{sq}}}{\partial b_2}(\boldsymbol{\theta}) = J_2'\big(a_2(x)\big)\,\frac{\partial a_2(x)}{\partial b_2} = J_2'\big(a_2(x)\big)\cdot 1$$

$$\frac{\partial \mathcal{L}_{\text{sq}}}{\partial w_2}(\boldsymbol{\theta}) = J_2'\big(a_2(x)\big)\,\frac{\partial a_2(x)}{\partial w_2} = J_2'\big(a_2(x)\big)\cdot \phi_1(x)$$

We still need the shared factor $J_2'\big(a_2(x)\big)$ in both expressions.

Reusing the output derivative

We need

$$J_2'\big(a_2(x)\big)$$

where

$$J_2(u) = J_3(\mathrm{ReLU}(u)w_3 + b_3)$$

Using the chain rule (when $a_2(x) \neq 0$)

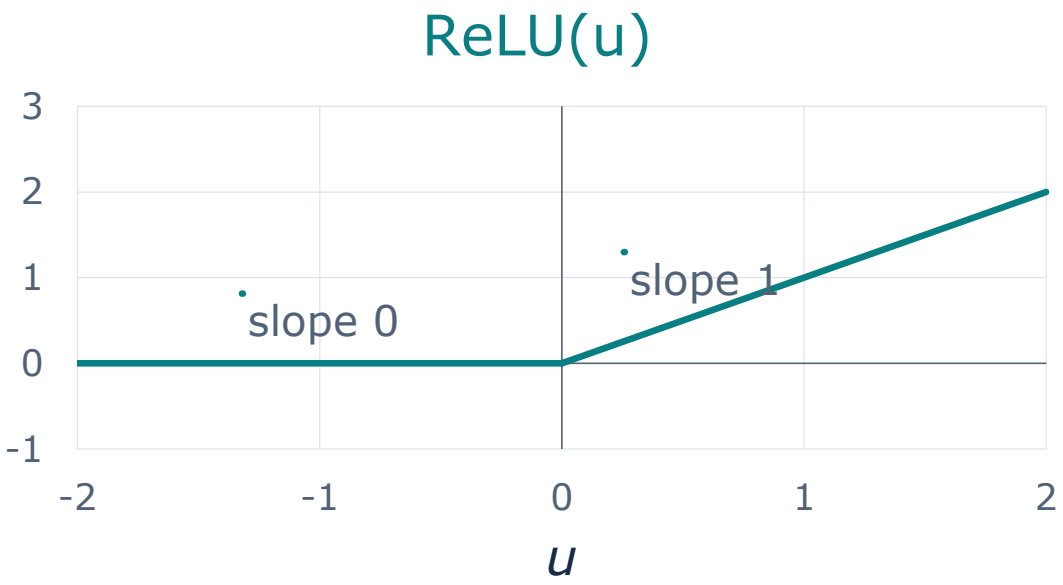
$$J_2'(u) = J_3'(\mathrm{ReLU}(u)w_3 + b_3) \cdot w_3 \cdot \mathrm{ReLU}'(u), \quad \text{when } u \neq 0$$

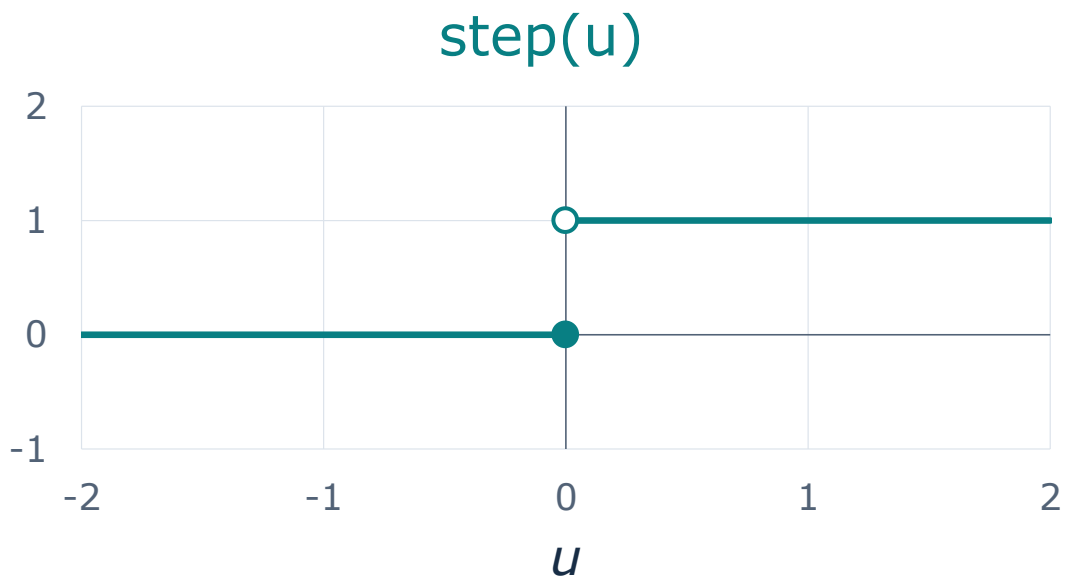
$$J_2'\big(a_2(x)\big) = J_3'(\underbrace{\mathrm{ReLU}\big(a_2(x)\big)w_3 + b_3}_{=a_3(x)}) \cdot w_3 \cdot \mathrm{ReLU}'\big(a_2(x)\big)$$

$$= \underbrace{J_3'\big(a_3(x)\big)}_{\substack{\text{already calculated} \\ \text{from the 3rd layer}}} \cdot \underbrace{w_3}_{\substack{\text{known} \\ \text{weight}}} \cdot \underbrace{\mathrm{ReLU}'\big(a_2(x)\big)}_{\text{still needed}}$$

The derivative of ReLU

The remaining factor is $\mathrm{ReLU}'(a_2(x))$, with $a_2(x) \neq 0$.





$$\mathrm{ReLU}'(u) = \mathrm{step}(u), \quad u \neq 0$$

$$\mathrm{step}(u) = \begin{cases} 1, & u > 0, \\ 0, & u \leq 0. \end{cases}$$

ReLU has no derivative at zero.
We choose $\mathrm{step}(0) = 0$ as a computational convention.

Completing the second-layer derivatives

We can now compute $\mathrm{ReLU}'\big(a_2(x)\big)$, with $a_2(x) \neq 0$.

$$J_2'\big(a_2(x)\big) = J_3'\big(a_3(x)\big) \cdot w_3 \cdot \mathrm{ReLU}'\big(a_2(x)\big)$$

$$= J_3'\big(a_3(x)\big) \cdot w_3 \cdot \mathrm{step}\big(a_2(x)\big)$$

Hence, we have computed

$$\frac{\partial \mathcal{L}_{\mathrm{sq}}}{\partial b_2}(\boldsymbol{\theta}) = J_2'\big(a_2(x)\big)$$

$$\frac{\partial \mathcal{L}_{\mathrm{sq}}}{\partial w_2}(\boldsymbol{\theta}) = J_2'\big(a_2(x)\big)\,\phi_1(x)$$

We will reuse $J_2'\big(a_2(x)\big)$ to calculate $J_1'\big(a_1(x)\big)$.

The chain rule at the first layer

Let's move to the 1st layer

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = J_1\big(a_1(x)\big)$$

where

$$J_1(u) = J_2(\text{ReLU}(u)w_2 + b_2) \qquad a_1(x) = \phi_0(x)w_1 + b_1$$

Neither J_1 nor $\phi_0(x)$ depends on w_1 or b_1

$$\frac{\partial \mathcal{L}_{\text{sq}}}{\partial b_1}(\boldsymbol{\theta}) = J_1'\big(a_1(x)\big)\,\frac{\partial a_1(x)}{\partial b_1} = J_1'\big(a_1(x)\big)\cdot 1$$

$$\frac{\partial \mathcal{L}_{\text{sq}}}{\partial w_1}(\boldsymbol{\theta}) = J_1'\big(a_1(x)\big)\,\frac{\partial a_1(x)}{\partial w_1} = J_1'\big(a_1(x)\big)\cdot \phi_0(x)$$

We still need the shared factor $J_1'\big(a_1(x)\big)$ in both expressions.

Reusing the derivative for the first layer

We need

$$J_1'\big(a_1(x)\big)$$

where

$$J_1(u) = J_2(\mathrm{ReLU}(u)w_2 + b_2)$$

Using the chain rule (when $a_1(x) \neq 0$ and $a_2(x) \neq 0$)

$$J_1'\big(a_1(x)\big) = J_2'(\underbrace{\mathrm{ReLU}\big(a_1(x)\big)w_2 + b_2}_{a_2(x)}) \cdot w_2 \cdot \mathrm{ReLU}'\big(a_1(x)\big)$$

$$= \underbrace{J_2'\big(a_2(x)\big)}_{\text{already calculated from the 2nd layer}} \cdot \underbrace{w_2}_{\text{known weight}} \cdot \underbrace{\mathrm{step}\big(a_1(x)\big)}_{\text{ReLU derivative}}$$

At zero, use $\mathrm{step}(0) = 0$ as a computational convention.

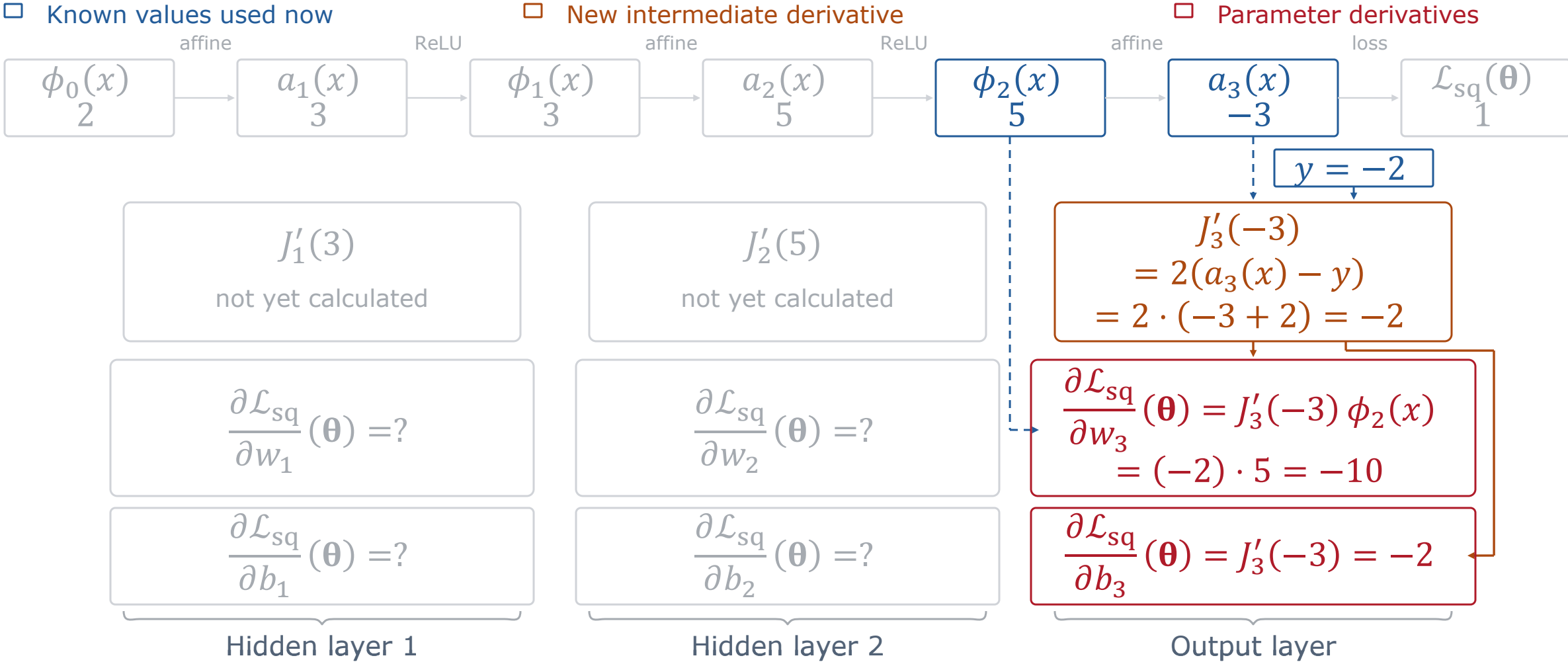
$$\frac{\partial \mathcal{L}_{\mathrm{sq}}}{\partial b_1}(\boldsymbol{\theta}) = J_1'\big(a_1(x)\big)$$

$$\frac{\partial \mathcal{L}_{\mathrm{sq}}}{\partial w_1}(\boldsymbol{\theta}) = J_1'\big(a_1(x)\big)\,\phi_0(x)$$

Backward pass: the output layer

Use the saved forward values to calculate the derivatives for layer 3.

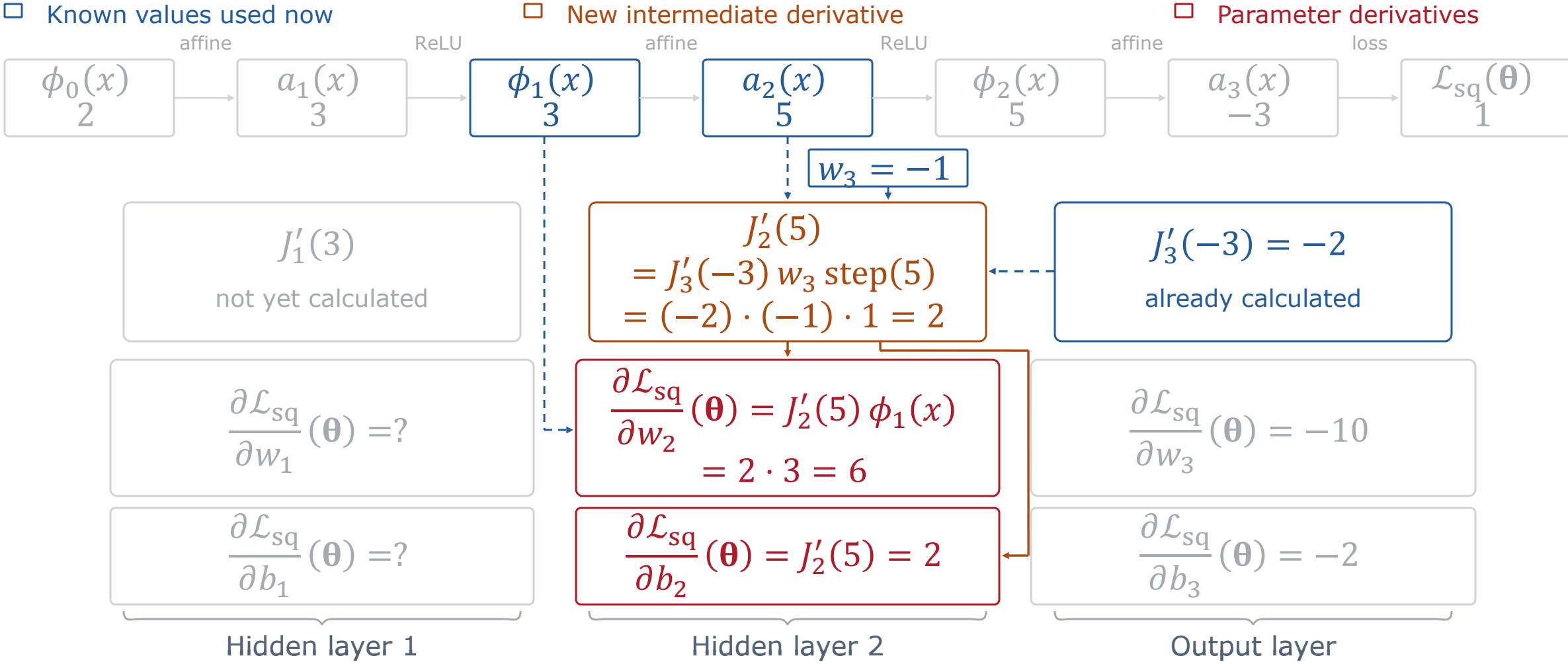
We will reuse $J_3'(-3) = -2$ to calculate $J_2'(5)$.



Backward pass: hidden layer 2

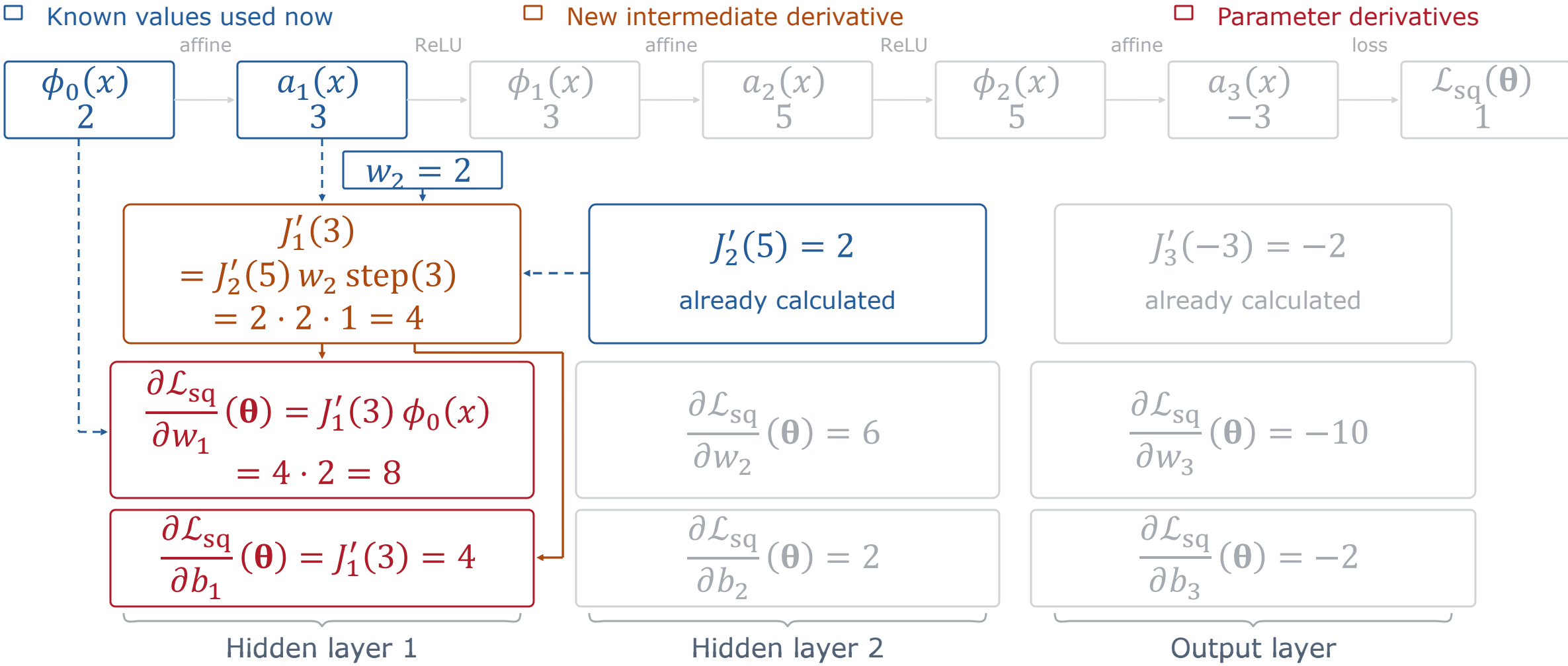
Reuse the derivative from layer 3 and the saved forward values for layer 2.

We will reuse $J_2'(5) = 2$ to calculate $J_1'(3)$.



Backward pass: hidden layer 1

Reuse the derivative from layer 2 and the saved forward values for layer 1.



We now have the complete gradient

Collect the derivatives in the same order as the parameter column.

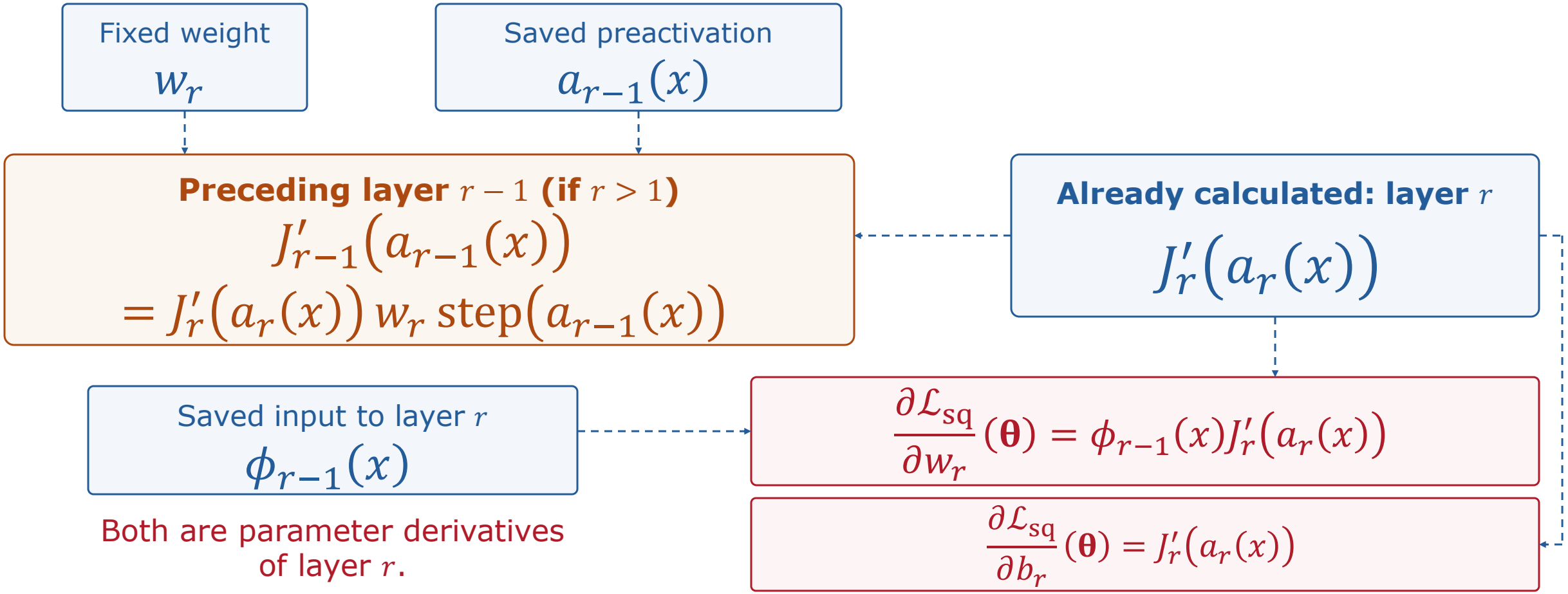
$$\nabla \mathcal{L}_{\text{sq}}\left(\boldsymbol{\theta}^{(t)}\right) = (4,8,2,6,-2,-10)^{\mathsf{T}}$$

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta(4,8,2,6,-2,-10)^{\mathsf{T}}$$

Only now do we update the parameters.

A general backward step: scalar network

Reuse the derivative already calculated for layer r.



The orange result becomes the known derivative at the next step.

In this lecture…

How can we efficiently calculate the derivatives of the empirical loss with respect to every weight and bias of a neural network?

Backpropagation in a scalar network

The same calculation with vectors

Extensions and computational cost

The forward pass for wider layers

The same forward calculation, with one coordinate per neuron.

Weights

$$\mathbf{W}_r \in \mathbb{R}^{d_{r-1} \times d_r},$$

Bias

$$\mathbf{b}_r \in \mathbb{R}^{1 \times d_r}$$

$$\boldsymbol{\phi}_0(\mathbf{x}) = \mathbf{x} \in \mathbb{R}^{1 \times d_0}$$

Preactivations

$$\mathbf{a}_r(\mathbf{x}) = \boldsymbol{\phi}_{r-1}(\mathbf{x})\mathbf{W}_r + \mathbf{b}_r, \qquad r = 1, \ldots, L$$

Activations

$$\boldsymbol{\phi}_r(\mathbf{x}) = \mathbf{ReLU}\big(\mathbf{a}_r(\mathbf{x})\big), \qquad r = 1, \ldots, L-1$$

The final affine row is the network output; no ReLU follows it.

The remaining derivative becomes a row

The remaining function still returns one loss; its input now has several coordinates.

Scalar network

$$J_r : \mathbb{R} \longrightarrow \mathbb{R}$$

Wider network

$$J_r : \mathbb{R}^{1 \times d_r} \longrightarrow \mathbb{R}$$

Squared loss, one output: $J_L(\mathbf{u}) = (u_1 - y)^2$

$$J_r(\mathbf{u}) = J_{r+1}(\mathbf{ReLU}(\mathbf{u})\mathbf{W}_{r+1} + \mathbf{b}_{r+1}), \quad r < L$$

$$J_r'\big(a_r(x)\big) \longrightarrow \nabla J_r\big(\mathbf{a}_r(\mathbf{x})\big)$$

$$\nabla J_r\big(\mathbf{a}_r(\mathbf{x})\big) = \left(\frac{\partial J_r}{\partial u_1}\big(\mathbf{a}_r(\mathbf{x})\big), \ldots, \frac{\partial J_r}{\partial u_{d_r}}\big(\mathbf{a}_r(\mathbf{x})\big)\right)$$

One partial derivative per input coordinate: a row of shape $1 \times d_r$.

Two different products

The same two matrices give different results under the two operations.

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad \mathbf{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

Matrix product

$$\mathbf{A} \cdot \mathbf{B} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

$$1 \cdot 5 + 2 \cdot 7 = 19$$

Componentwise product

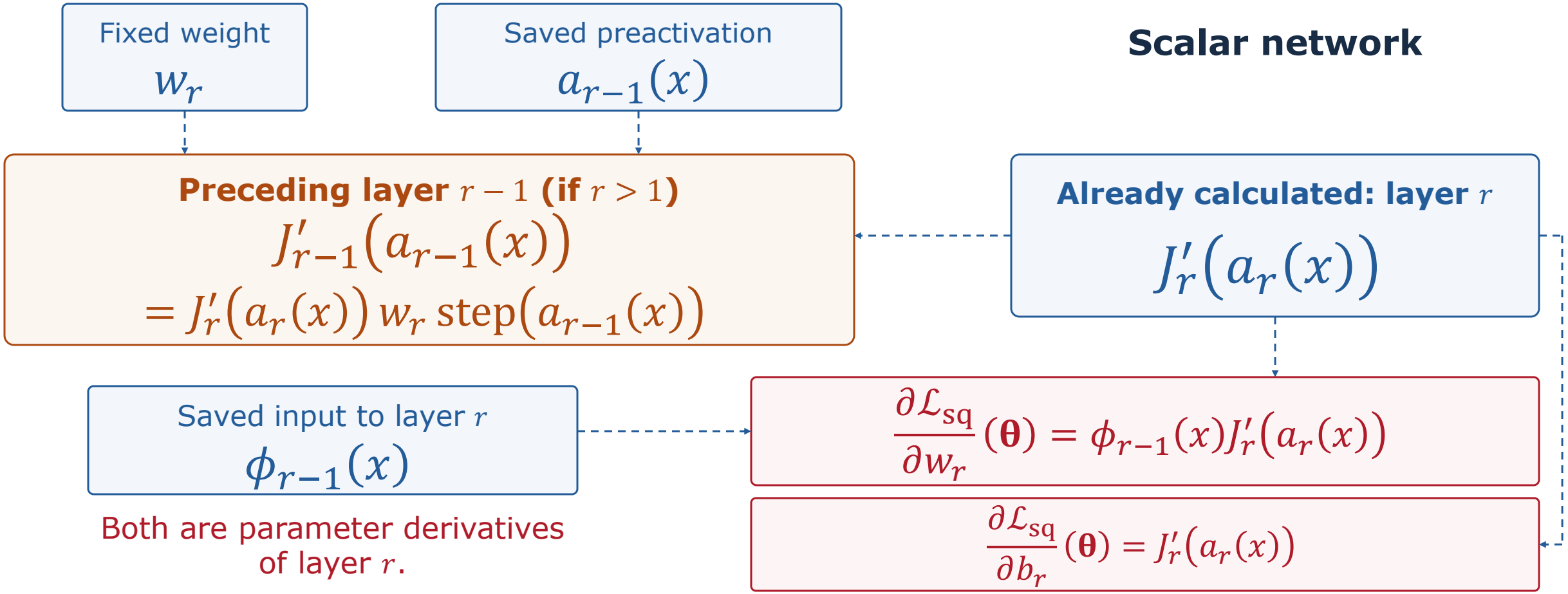
$$\mathbf{A} \odot \mathbf{B} = \begin{pmatrix} 5 & 12 \\ 21 & 32 \end{pmatrix}$$

$$1 \cdot 5 = 5$$

The operands are the same. The operation changes.

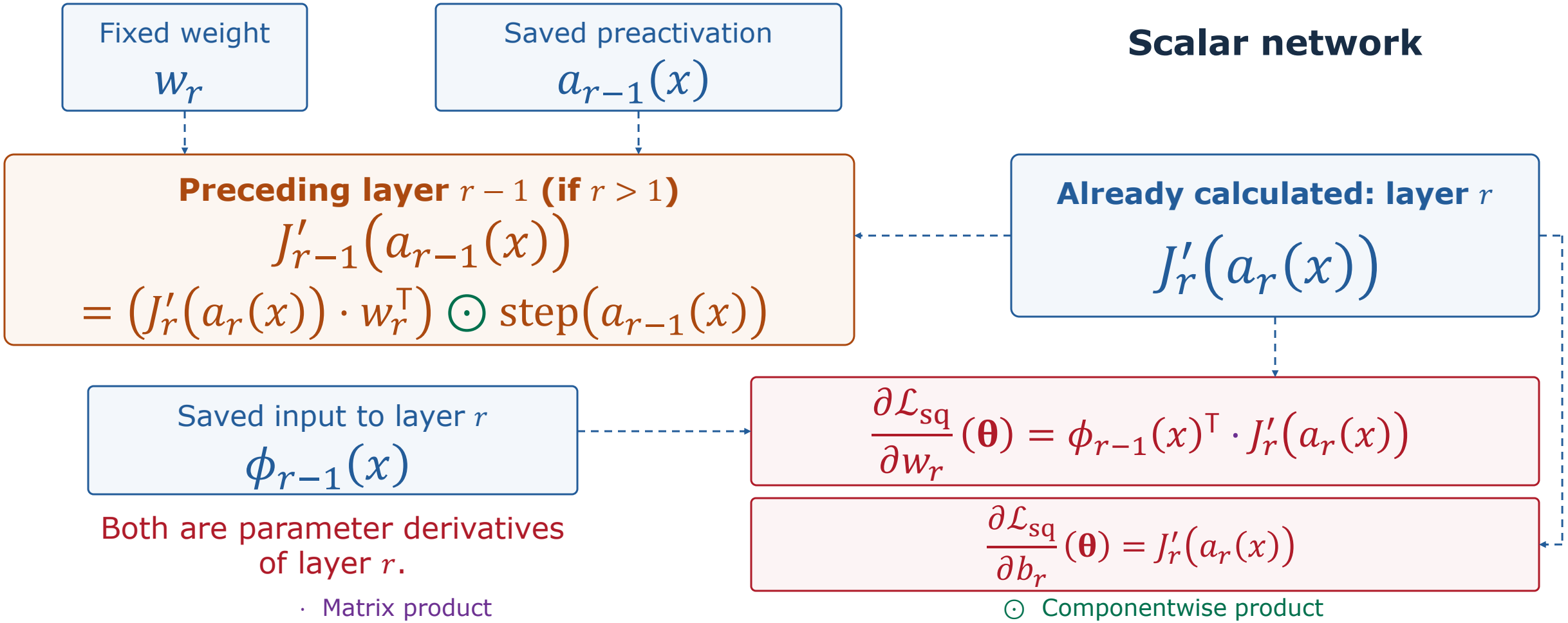
A general backward step: scalar network

Recall the scalar backward step: every quantity is a number.
All products are ordinary scalar multiplications.



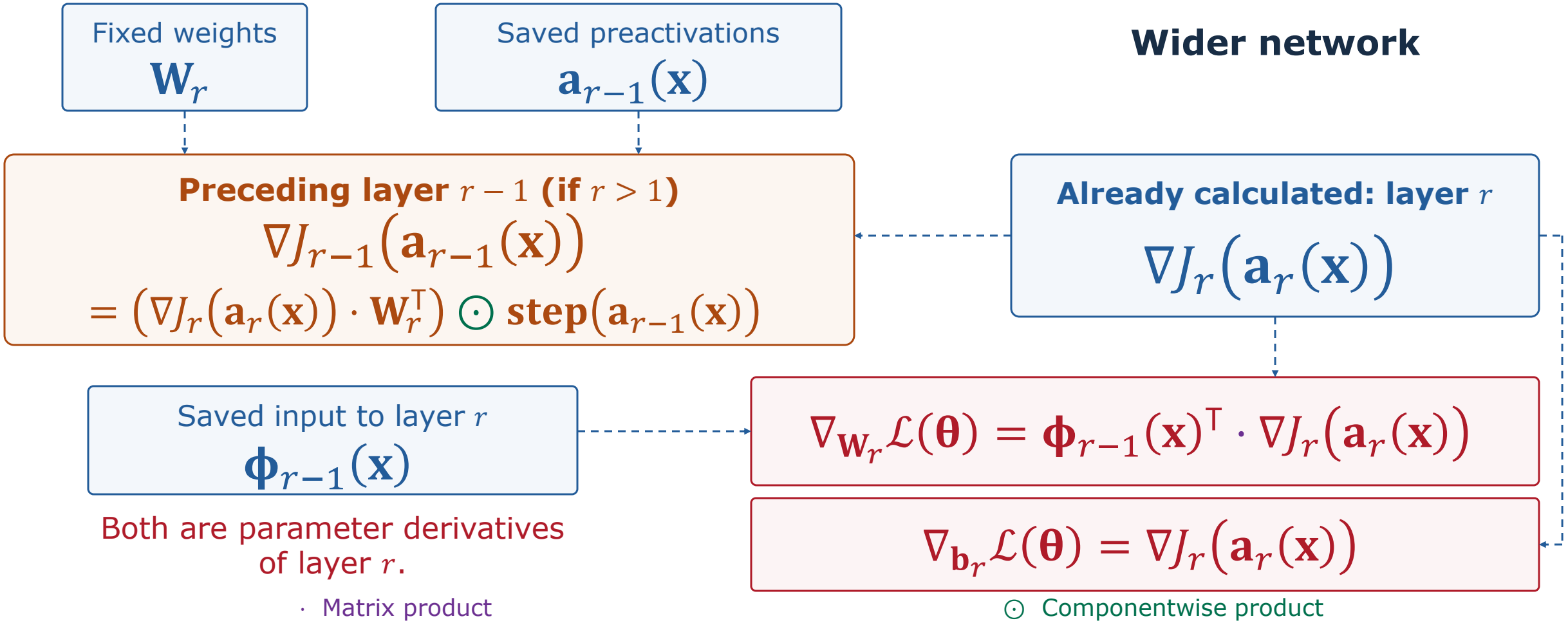
A general backward step: scalar network

For the scalar diagram, identify each number with a 1×1 matrix.
Transposes leave it unchanged; both products multiply the entries.



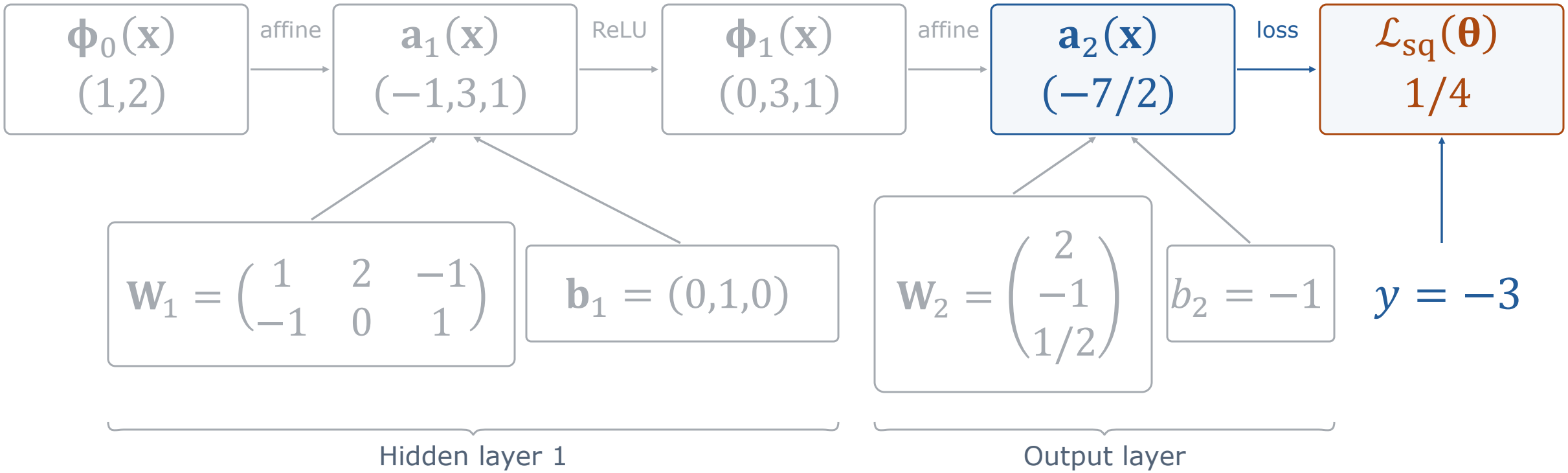
A general backward step: wider network

We write $\mathcal{L}$ for a general loss, differentiable with respect to the output.
Given its output gradient, the same backward rules apply.



Return to the network from Lecture 4

The same weights and input; now take the regression target $y = -3$.



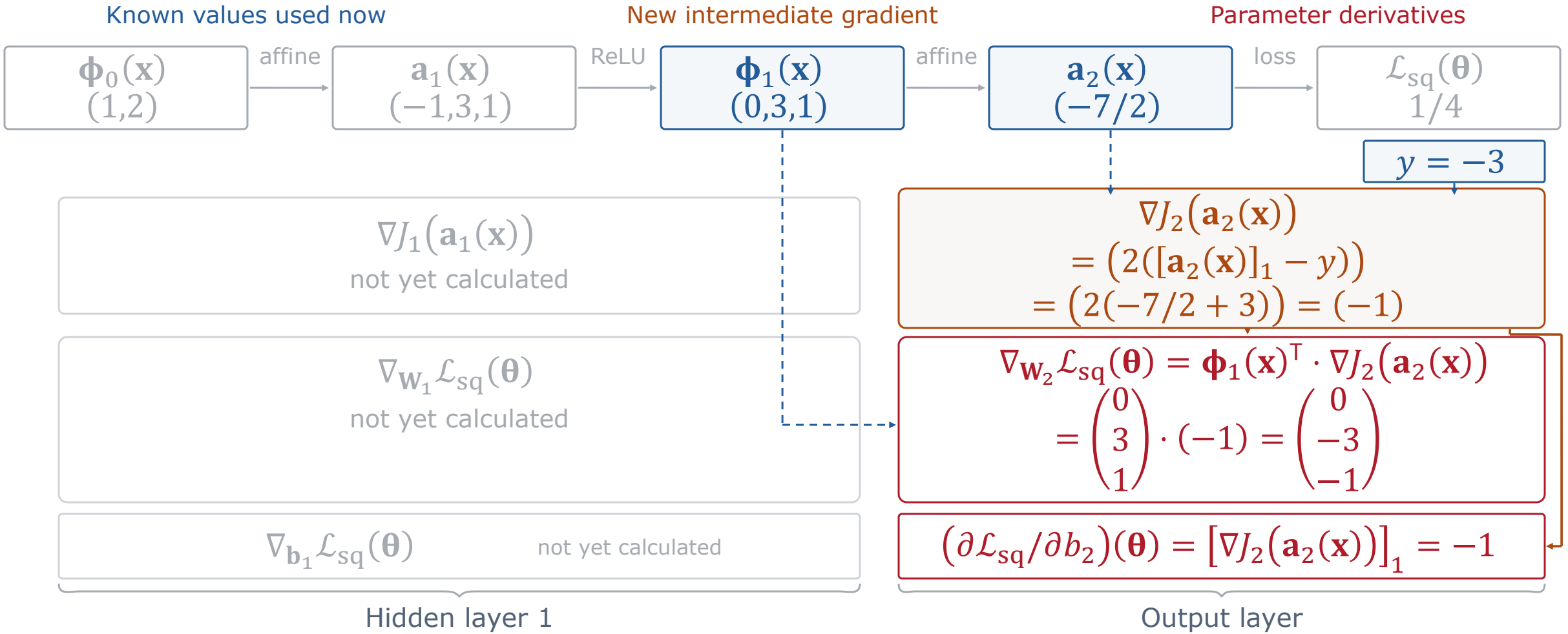
$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = ([\mathbf{a}_2(\mathbf{x})]_1 - y)^2 = (-7/2 + 3)^2 = 1/4$$

Keep these values: the backward pass will reuse them.

Backward pass: the output layer

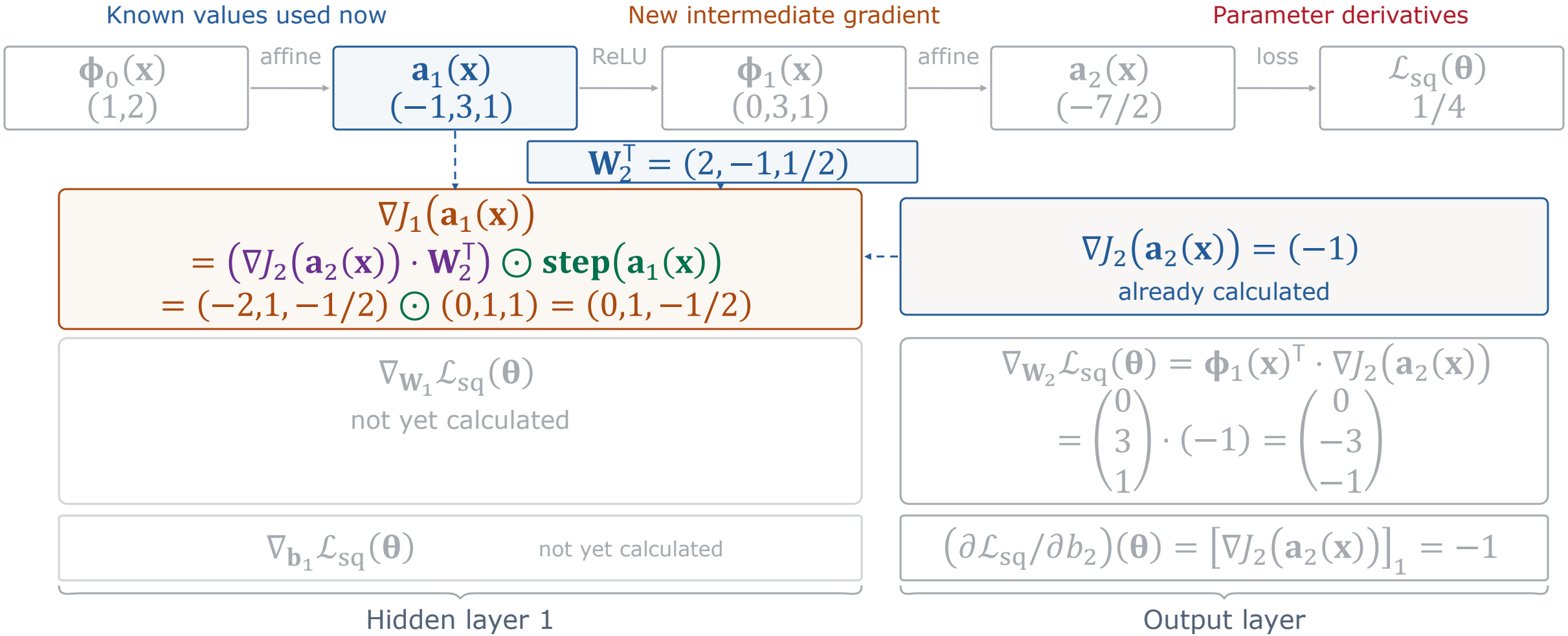
Use the saved forward values; the output gradient is a row of shape 1 × 1.

We will reuse $\nabla J_2(\mathbf{a}_2(\mathbf{x})) = (-1)$ to calculate $\nabla J_1(\mathbf{a}_1(\mathbf{x}))$.



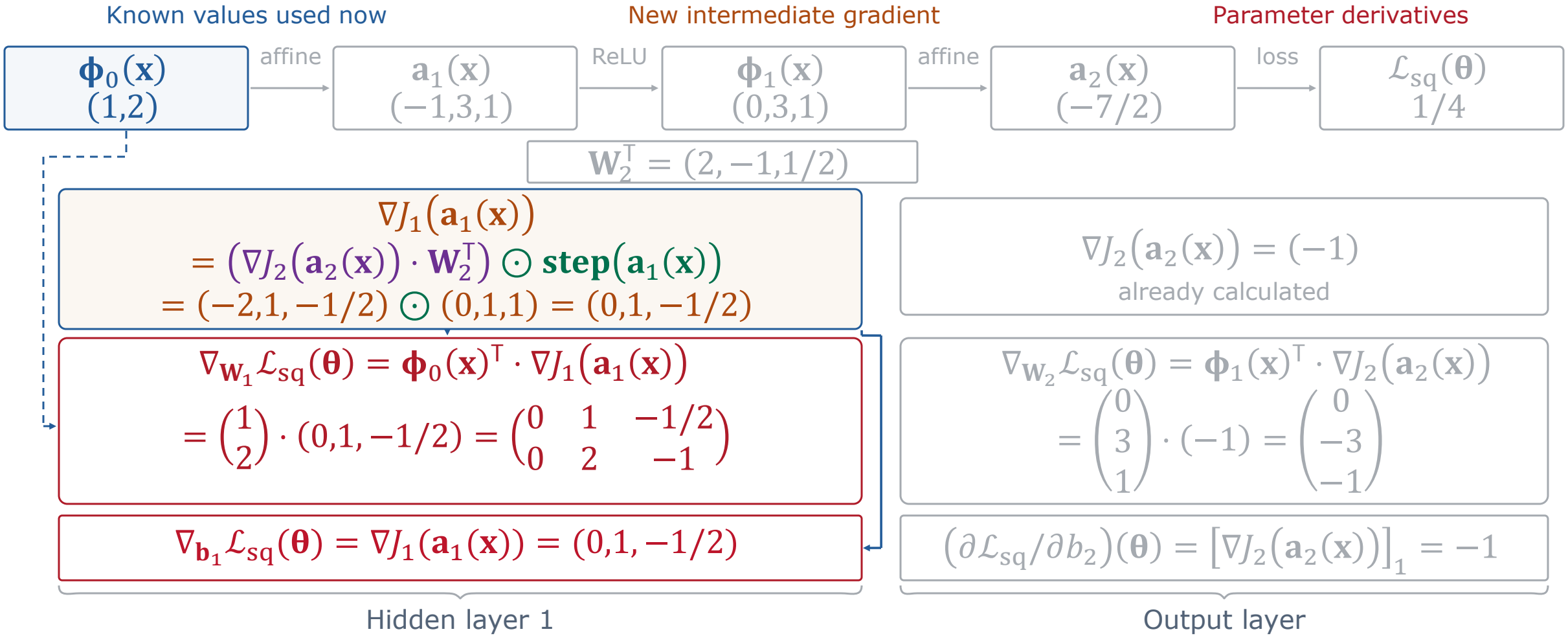
Backward pass: hidden layer 1

Reuse the output gradient with the saved hidden preactivations and fixed output weights.



The hidden-layer parameter gradients

Reuse $\nabla J_1(\mathbf{a}_1(\mathbf{x}))$ with the saved input $\boldsymbol{\phi}_0(\mathbf{x}) = (1,2)$.



In this lecture…

How can we efficiently calculate the derivatives of the empirical loss with respect to every weight and bias of a neural network?

Backpropagation in a scalar network

The same calculation with vectors

Extensions and computational cost

Regression vs Binary classification

Output layer width: $d_L = 1$

Regression

$$J_L(\mathbf{u}) = (u_1 - y)^2$$

$$\nabla J_L\big(\mathbf{a}_L(\mathbf{x})\big) \in \mathbb{R}^{1\times 1}$$

$$\big[\nabla J_L\big(\mathbf{a}_L(\mathbf{x})\big)\big]_1 = 2(f_{\boldsymbol{\theta}}(\mathbf{x}) - y)$$

Binary classification

$$J_L(\mathbf{u}) = \ell_{\log}(\sigma(u_1), y)$$

$$\nabla J_L\big(\mathbf{a}_L(\mathbf{x})\big) \in \mathbb{R}^{1\times 1}$$

$$\big[\nabla J_L\big(\mathbf{a}_L(\mathbf{x})\big)\big]_1 = p_{\boldsymbol{\theta}}(\mathbf{x}) - y$$

Where

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma\big(f_{\boldsymbol{\theta}}(\mathbf{x})\big) = \sigma([\underbrace{\mathbf{a}_L(\mathbf{x})}_{\in\mathbb{R}^{1\times 1}}]_1)$$

Multiclass: one initial derivative per score

Output layer width: $d_L = K$

$$\boldsymbol{\theta} \longmapsto \ell_{\text{multi}}(\mathbf{p}_{\boldsymbol{\theta}}(\mathbf{x}), y)$$

where

$$\mathbf{p}_{\boldsymbol{\theta}}(\mathbf{x}) = \left(p_{\boldsymbol{\theta},1}(\mathbf{x}), \ldots, p_{\boldsymbol{\theta},K}(\mathbf{x})\right) = \text{softmax}\left(\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x})\right) = \text{softmax}\left(\mathbf{a}_L(\mathbf{x})\right)$$

Initial derivative for backpropagation

$$\nabla J_L\left(\mathbf{a}_L(\mathbf{x})\right) = \left(p_{\boldsymbol{\theta},1}(\mathbf{x}), \ldots, p_{\boldsymbol{\theta},y}(\mathbf{x}) - 1, \ldots, p_{\boldsymbol{\theta},K}(\mathbf{x})\right)$$

For example:

$$K = 3, \qquad y = 2:$$

$$\nabla J_L(\mathbf{a}_L(\mathbf{x})) = \left(p_{\boldsymbol{\theta},1}(\mathbf{x}), p_{\boldsymbol{\theta},2}(\mathbf{x}) - 1, p_{\boldsymbol{\theta},3}(\mathbf{x})\right)$$

From one observation to the whole dataset

The empirical loss is the sum of the individual losses.

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \mathcal{L}_i(\boldsymbol{\theta})$$

Each term is the loss for one observation, using the chosen task.

$$\nabla\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \nabla\mathcal{L}_i(\boldsymbol{\theta})$$

Every summand is evaluated at the same current parameters.

One complete gradient-descent update

1. Start with a zero gradient accumulator.

2. For each observation:

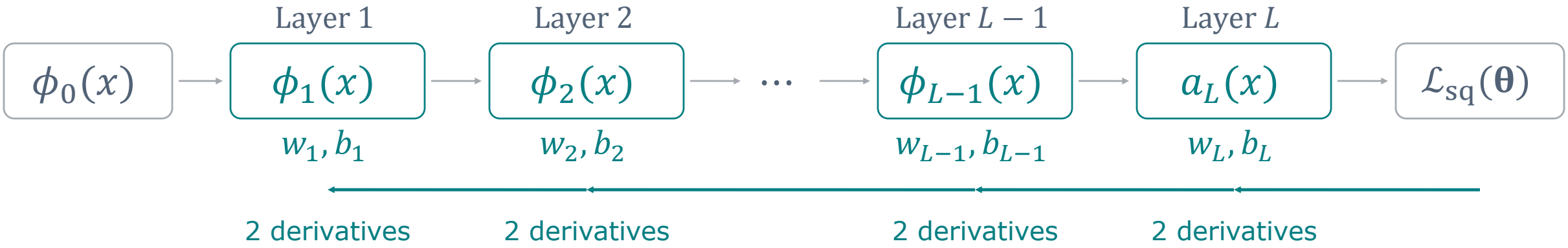
Forward pass → backward pass → add its parameter derivatives.

3. Use the accumulated gradient to update all parameters.

Why does reuse save work?

In a scalar network with L layers, there are $P = 2L$ parameters.

Backpropagation



Compute each intermediate derivative once.

Reuse it for both parameter derivatives, then continue backward.

One forward pass + one backward pass

$$O(P)$$

Take-home messages from backpropagation

Its purpose is to calculate the gradient efficiently.

Forward: save preactivations and inputs to each layer.

Backward: reuse derivatives, layer by layer.

Update: change parameters using the computed gradient.

Lecture notes and exercises

The notes for today's lecture are available on the course GitHub page.

They explain the material in more detail and include exercises.

I will post the exercise solutions before the next lecture.

github.com/semtm0051/2026_27

Scan for course materials